

Presenton — Skill de Usabilidade (MCP de Apresentações IA, Self-Hosted)

Para qual Agente serve

Portinari (Apresentações) — equipe de Lúcio (Gestor Arquitetura) do Sistema Orgânico STTK. Esta ferramenta complementa o PPTAgent (documentado em 26/08): onde PPTAgent é CLI-only para geração de PPTX, **Presenton tem servidor MCP integrado** — Claude (como Portinari) pode gerar apresentações diretamente via protocolo MCP, sem CLI intermediário.

Status

proposta — aguardando implantação (Drenagem ainda não confirmou)

O que a ferramenta faz

Presenton é um gerador de apresentações por IA com: - **Servidor MCP integrado** — permite que Claude interaja diretamente com a ferramenta via protocolo MCP para criar e editar apresentações - **Interface web/desktop** — app completo com UI visual (alternativa ao Gamma/Canva, self-hosted) - **Múltiplos provedores de modelo** — usa Claude, OpenAI, Gemini, Vertex AI, **Ollama (gratuito local)**, LM Studio, ou qualquer OpenAI-compatible API - **Desktop app** (Electron): todo processamento local, sem dependência de nuvem - **Docker**: deploy self-hosted em servidor

Diferença do PPTAgent (26/08): PPTAgent gera PPTX via CLI/API usando HuggingFace. Presenton tem servidor MCP nativo — Portinari pode gerar slides diretamente sem arquivo intermediário.

Como se usa

Opção 1 — Desktop App (recomendado para uso imediato)

```
# Instalar (Node.js LTS + Python 3.11 + uv requeridos)
git clone https://github.com/presenton/presenton
cd presenton/servers/nextjs
npm install
npm run build && npm start
```

Abrir `localhost:3000` no browser. Configurar provedor: selecionar **Ollama** com modelo local (gratuito).

Opção 2 — Docker (self-hosted em servidor)

```
docker pull presenton/presenton
docker run -p 3000:3000 presenton/presenton
```

Opção 3 — MCP Server para Claude/Portinari

```
// Adicionar em settings.json do Claude Code:
{
  "mcpServers": {
    "presenton": {
      "command": "npx",
      "args": ["presenton-mcp"],
      "env": {
        "PRESENTON_URL": "http://localhost:3000"
      }
    }
  }
}
```

Fluxo MCP: Claude (como Portinari) → Presenton MCP → app Presenton local → apresentação gerada.

Requisitos Técnicos

- Node.js LTS (requerido para desktop)
- Python 3.11 + uv (requerido para desktop)
- Ollama instalado localmente (para uso 100% gratuito, sem API key)
- OU qualquer modelo API compatível com OpenAI (Anthropic/Claude aceito)
- Disco: 80 MB+ (app) + tamanho do modelo Ollama (ex: Llama 3.2 = ~2 GB)

Evidência de segurança (Princípio 3)

- **Custo:** Zero — usando Ollama com modelo local (ex: Llama 3.2, Gemma 2). Sem mensalidade, sem API paga obrigatória.
- **Vazamento de dado de cliente:** Desktop app processa localmente ("All processing happens on your device, no cloud dependencies"). Docker self-hosted: zero upload externo se configurado com Ollama local.
- **Idoneidade:** Apache 2.0 · 9.9k stars · 1.5k forks · 2.692 commits no main · repositório ativo (github.com/presenton/presenton) · sem sinal de typosquatting, sem referência a malware
- **Alternativa a:** Gamma, Canva, Beautiful.ai, Decktopus (todos pagos/SaaS) — Presenton é a alternativa open-source verificada

Limitações honestas

- Requer Node.js + Python 3.11 instalados (setup inicial de ~15 min)
- Qualidade das apresentações geradas depende do modelo escolhido — Ollama local pode ser menos polido que GPT-4/Claude

- MCP server do Presenton é componente relativamente novo — verificar se está estável antes de implantar em produção
- Desktop app é Electron (Windows/Mac/Linux) — não é web puro; requer instalação local

Fonte

- GitHub: github.com/presenton/presenton — Apache 2.0, 9.9k stars, 1.5k forks (verificado por WebFetch em 28/08/2026)
- Data de verificação: 28/08/2026